

GCP Data Engineer Complete Q&A

Compiled from Altimetrik · LTI Mindtree · Tech Mahindra · HCL · Deloitte · Quantiphi · HSBC

Created by **Mantu Kumar Deka**

CHAPTER 01 GCP Cloud Services & Architecture

Q What are the core GCP services used in a Data Engineering pipeline?

SERVICE	PURPOSE
Cloud Storage (GCS)	Raw/staging file storage (CSV, JSON, Parquet, Avro)
BigQuery (BQ)	Serverless data warehouse for analytics
Pub/Sub	Real-time messaging / event streaming
Dataflow	Managed Apache Beam — batch & streaming ETL
Dataproc	Managed Spark/Hadoop cluster
Cloud Composer	Managed Apache Airflow for workflow orchestration
Cloud Functions	Serverless event-driven code (triggers on GCS, Pub/Sub)
Cloud Run	Containerised microservices
Secret Manager	Secure storage of credentials & API keys
IAM	Identity & Access Management — roles, policies
Vertex AI	ML model training & serving

Q Explain BigQuery Architecture.

BigQuery uses a **serverless, columnar storage architecture** that separates compute and storage:

- **Storage Layer:** Data stored in Google's Colossus distributed file system as columnar format (Capacitor). Supports native tables, external tables, materialized views.
- **Compute Layer:** Dremel query engine executes queries using a tree architecture — root server → mixers → leaf nodes (read from Colossus).
- **Petabit Network:** Jupiter network connects compute to storage at massive bandwidth.
- **Slots:** A slot = unit of computational capacity. On-demand: 2000 slots per project.
- **Pricing Models:** On-demand (per TB scanned) or Capacity (flat-rate slot reservations).

Q What is the difference between Dataflow and Dataproc?

FEATURE	DATAFLOW	DATAPROC
Engine	Apache Beam	Apache Spark / Hadoop
Management	Fully serverless — no cluster mgmt	Managed cluster — you control nodes
Scaling	Auto-scales workers dynamically	Manual or auto-scaling of cluster
Best for	Streaming + batch ETL pipelines	Existing Spark/Hadoop workloads
Cost model	Pay per vCPU/memory/time	Pay for cluster uptime
Startup time	Slower (pipeline compilation)	Faster (cluster pre-exists)

Q What are GCS Storage Classes?

STORAGE CLASS	ACCESS PATTERN	USE CASE	MIN DURATION
Standard	Frequently accessed	Hot data, active workloads	None
Nearline	~Once a month	Backups, monthly reports	30 days
Coldline	~Once per 90 days	Disaster recovery	90 days
Archive	Rarely accessed	Long-term archival	365 days

Note: Maximum object size in GCS = 5 TB

Q Why do we need a GCS bucket (staging area)?

- Decouples ingestion from processing — source systems dump files independently.
- Serves as a buffer / landing zone before transformation.
- Enables schema validation before loading into BigQuery.
- Allows reprocessing of raw data without re-extracting from source.
- Supports cost-effective storage of raw/historical data.
- BigQuery can load directly from GCS (native integration).

Q What is Pub/Sub and where is it used?

Pub/Sub is a fully managed, real-time messaging service for decoupled event-driven architectures.

FEATURE	PUSH	PULL
How	Pub/Sub pushes to endpoint URL	Subscriber explicitly calls Pull API
Use case	Cloud Functions, webhooks	Dataflow, Dataproc consumers
Scaling	Auto	Manual ACK required

Q What are Dataflow Window types?

- **Fixed/Tumbling:** Non-overlapping windows of fixed duration (e.g., every 5 min).
- **Sliding:** Overlapping windows (e.g., 10-min window every 2 min).
- **Session:** Dynamic windows based on gaps in activity.
- **Global:** Single window covering all elements.

ParDo: Core parallel processing transform in Apache Beam — applies a DoFn function to each element.

Q How to recover deleted data in BigQuery?

- **Time Travel:** BQ retains historical data for up to 7 days. Query with `FOR SYSTEM_TIME AS OF`.
- **Table Snapshot / Clone:** Restore from a snapshot taken before deletion.
- **Fail-safe:** Additional 7-day window (not queryable, but Google support can assist).

●SQL — BigQuery Time Travel

```
SELECT *
FROM `project.dataset.table`
FOR SYSTEM_TIME AS OF TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 1 HOUR);
```

Q What is CI/CD and how does it work in GCP pipelines?

CI/CD automates build, test, and deploy across environments.

- **GitHub / Cloud Source Repositories:** Source code management.
- **Cloud Build:** Triggers on git push, runs unit tests, builds Docker images.
- **Artifact Registry:** Stores Docker images.
- **Terraform / Deployment Manager:** Infrastructure as Code.
- **Cloud Deploy:** Manages deployment to staging/prod environments.

●Bash — Git Workflow

```
# Create feature branch
git checkout -b feature/my-branch

# Stage and commit changes
git add .
git commit -m 'feat: add transform'

# Push and create Pull Request
git push origin feature/my-branch
# PR → review → merge to main → Cloud Build triggers automatically
```

What is the Pub/Sub lifecycle and retention period?

Q What is the Pub/Sub lifecycle and retention period?

Message lifecycle: Published → stored in topic → delivered to subscriptions → subscriber ACKs → message deleted.

STAGE	DESCRIPTION
Publish	Producer sends message to a Topic
Store	Message stored durably in Pub/Sub backend
Deliver	Forwarded to all Subscriptions on the topic
Acknowledge	Subscriber sends ACK within ack_deadline (default 10s-600s)
Delete	Acknowledged message removed from subscription
Expire	Unacknowledged message deleted after retention period

- **Default retention:** 7 days. Configurable from 10 minutes to **31 days**.
- **Dead-letter topic:** Messages that fail delivery after max attempts are forwarded here.
- **At-least-once delivery:** Pub/Sub may deliver duplicates — consumers must be idempotent.

Q How do you choose a GCS storage class?

Choose based on **access frequency** — lower storage cost comes with higher retrieval cost:

CLASS	ACCESS PATTERN	MIN DURATION	BEST FOR
Standard	Daily / frequent	None	Active pipeline data, hot data
Nearline	~Once a month	30 days	Monthly backups, reports
Coldline	~Once per 90 days	90 days	Disaster recovery, quarterly audits
Archive	Rarely (once a year)	365 days	Long-term compliance archival

Tip: Use **lifecycle rules** to auto-transition objects Standard → Nearline → Coldline → Archive as they age.

Is Nearline cheaper if accessed once a month? Only if truly ~once/month. Higher access frequency makes Standard cheaper overall due to per-retrieval costs.

Q What is Cloud Functions?

Cloud Functions is a **serverless, event-driven** execution platform — no infrastructure to manage.

- **Triggers:** GCS object creation, Pub/Sub message, HTTP request, Firestore event.
- **Use cases:** Auto-trigger ETL when a file lands in GCS; lightweight transformations; send alerts; invoke APIs on events.
- **vs Cloud Run:** Cloud Functions is for single-purpose lightweight functions; Cloud Run supports containerised services with longer runtimes.

Q What is partitioning and clustering in BigQuery?

Partitioning divides a table into segments based on a column value so BQ scans only relevant partitions.

Clustering sorts data within each partition by one or more columns for block-level pruning.



●SQL — Create Partitioned & Clustered Table

```
CREATE TABLE sales.orders
PARTITION BY DATE(order_date)
CLUSTER BY region, product_id
AS SELECT * FROM raw.orders;
```

FEATURE	PARTITIONING	CLUSTERING
Granularity	Partition level	Block level within partition
Max columns	1	4
Cost savings	Eliminates entire partitions	Prunes blocks within partitions
Best for	Date/time range filters	High-cardinality filter columns

Q What are Row-level and Column-level security policies in BigQuery?



●SQL — Row Access Policy

```
CREATE ROW ACCESS POLICY finance_only
ON dataset.table
GRANT TO ('group:finance@company.com')
FILTER USING (department = 'Finance');
```

Column-level Security (Policy Tags): Tag sensitive columns (PII, salary) via Data Catalog taxonomy. Users without access see NULL for those columns.

IAM Roles: `roles/bigquery.dataViewer`, `dataEditor`, `dataOwner`, `jobUser`, `admin`

Q Standard View vs Materialized View

FEATURE	STANDARD (LOGICAL) VIEW	MATERIALIZED VIEW
Storage	No physical storage	Physically stores pre-computed results
Performance	Slower — re-executes SQL every time	Faster — reads cached results
Freshness	Always fresh	Refreshes on schedule or on base DML
Cost	Billed per query on base tables	Storage cost + refresh cost
Use case	Access control, simplify complex queries	Aggregations queried frequently

Q What does QUALIFY mean in BigQuery?

QUALIFY filters results of window functions — similar to HAVING for aggregate functions.

●●●

●SQL — QUALIFY for Deduplication

```
-- Keep only the latest record per user
SELECT *
FROM events
QUALIFY ROW_NUMBER() OVER (
    PARTITION BY user_id
    ORDER BY event_time DESC
) = 1;
```

Q How to implement SCD Type 2 in BigQuery?

SCD Type 2 tracks historical changes by adding new rows with effective date and a current flag.

●●●

●SQL — SCD Type 2 MERGE

```
MERGE dim_customer AS target
USING staging_customer AS source
    ON target.customer_id = source.customer_id
    AND target.is_current = TRUE

WHEN MATCHED AND (
    target.email != source.email
    OR target.address != source.address
) THEN UPDATE SET
    target.is_current = FALSE,
    target.end_date = CURRENT_DATE()

WHEN NOT MATCHED THEN INSERT
    (customer_id, email, address, start_date, end_date, is_current)
VALUES
    (source.customer_id, source.email, source.address,
    CURRENT_DATE(), NULL, TRUE);
```

Q How is BigQuery different from RDBMS?

FEATURE	RDBMS (E.G., POSTGRESQL)	BIGQUERY
Storage	Row-based storage	Columnar storage (Capacitor)
Scaling	Vertical (scale up)	Horizontal, serverless auto-scale
Transactions	Full ACID support	Limited DML (MERGE, INSERT, UPDATE)
Indexes	B-tree, hash indexes	No traditional indexes — uses partitioning & clustering
Constraints	Enforced (PK, FK, UNIQUE)	Informational only — not enforced
Best for	OLTP — frequent small reads/writes	OLAP — large analytical queries
Cost model	Instance/licence based	Pay per TB scanned or slot reservation

Q What is an Authorized View in BigQuery?

An **Authorized View** lets you share query results with users without giving them direct access to the underlying base tables.

- Grant the view itself access to the source dataset (not the user).
- Users query the view — BigQuery uses the view's permissions to read source data.
- Ideal for exposing only non-sensitive columns to BI tools or external teams.
- No data duplication — view runs live against source tables.

●SQL — Create Authorized View

```
-- 1. Create the view in a separate dataset
CREATE VIEW reporting.emp_public_view AS
SELECT emp_id, name, dept    -- salary excluded
FROM   hr.employees;

-- 2. Authorize the view on the source dataset via BQ Console:
--    hr dataset → Share → Authorized Views → add reporting.emp_public_view
```

Q Query optimization techniques in BigQuery

- Use **partitioned tables** with partition filters — avoid full table scans.
- Use **clustered tables** — prunes blocks based on filter columns.
- **SELECT specific columns** — BQ is columnar; avoid `SELECT *`.
- **Filter early** — push WHERE clauses as early as possible.
- Use **Materialized Views** for frequently aggregated queries.
- Use `APPROX_COUNT_DISTINCT` instead of `COUNT(DISTINCT)` for large datasets.
- **Denormalize** — use STRUCT/ARRAY to avoid expensive JOINS.
- Use **BI Engine** for sub-second dashboard queries.

Q How to query STRUCT and ARRAY data types in BigQuery?

●SQL — STRUCT & ARRAY UNNEST

```
-- Query nested STRUCT fields
SELECT
  order_id,
  customer.name,
  customer.address.city
FROM orders;

-- Flatten ARRAY of STRUCTs using UNNEST
SELECT
  order_id,
  item.product_name,
  item.qty
FROM orders,
UNNEST(line_items) AS item;
```

Q SQL to remove duplicate rows (raw to history layer)

●SQL — Deduplication Methods

```
-- Method 1: ROW_NUMBER — keep latest record
INSERT INTO history_table
SELECT * EXCEPT(rn)
FROM (
    SELECT *,
        ROW_NUMBER() OVER (
            PARTITION BY emp_id
            ORDER BY updated_at DESC
        ) AS rn
    FROM raw_table
)
WHERE rn = 1;

-- Method 2: QUALIFY (BigQuery specific)
SELECT *
FROM raw_table
QUALIFY ROW_NUMBER() OVER (
    PARTITION BY emp_id
    ORDER BY updated_at DESC
) = 1;
```

Q Find 2nd and 3rd highest salary

●SQL — Nth Highest Salary

```
-- 2nd highest salary (simple)
SELECT DISTINCT salary
FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 1;

-- Using DENSE_RANK
SELECT emp_name, salary
FROM (
    SELECT emp_name, salary,
        DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
)
WHERE rnk = 2;

-- 2nd highest salary per department
SELECT dept_id, emp_name, salary
FROM (
    SELECT dept_id, emp_name, salary,
        DENSE_RANK() OVER (
            PARTITION BY dept_id
            ORDER BY salary DESC
        ) AS rnk
    FROM employees
)
WHERE rnk = 2;
```



```

    ) AS rnk
FROM employees
)
WHERE rnk = 2;

```

Q Window functions — RANK vs DENSE_RANK vs ROW_NUMBER

FUNCTION	TIES	GAP AFTER TIE?	USE CASE
ROW_NUMBER()	Each row unique number	N/A	Deduplication, pagination
RANK()	Ties get same rank	Yes (skips numbers)	Standard ranking with gaps
DENSE_RANK()	Ties get same rank	No (no gaps)	N-th highest value
NTILE(n)	Divides into n buckets	N/A	Percentile distribution
LAG/LEAD	Offset to prev/next row	N/A	Day-over-day change

Q Customers who bought continuously for 3 days


SQL — Consecutive 3-Day Buyers

```

WITH daily AS (
    SELECT customer_id,
           DATE(order_date) AS order_day
    FROM orders
    GROUP BY 1, 2
),
lagged AS (
    SELECT customer_id,
           order_day,
           LAG(order_day, 1) OVER (PARTITION BY customer_id ORDER BY order_day) AS prev1,
           LAG(order_day, 2) OVER (PARTITION BY customer_id ORDER BY order_day) AS prev2
    FROM daily
)
SELECT DISTINCT customer_id
FROM lagged
WHERE order_day = DATE_ADD(prev1, INTERVAL 1 DAY)
    AND order_day = DATE_ADD(prev2, INTERVAL 2 DAY);

```

Q Pipeline restart — upsert pattern (idempotent loads)


SQL — MERGE Upsert Pattern

```

MERGE target_table AS T
USING staging_table AS S
    ON T.emp_id = S.emp_id

WHEN MATCHED THEN UPDATE SET

```

```
T.name      = S.name,
T.salary    = S.salary,
T.updated_at = CURRENT_TIMESTAMP()

WHEN NOT MATCHED THEN INSERT
  (emp_id, name, salary, inserted_at)
VALUES
  (S.emp_id, S.name, S.salary, CURRENT_TIMESTAMP());
```

Note: This makes the pipeline idempotent — safe to restart without duplicate inserts.

Q **WHERE vs HAVING**

- **WHERE** — Filters rows *before* grouping. Cannot use aggregate functions.
- **HAVING** — Filters *after* grouping. Can use aggregate functions.

```
SELECT  dept, AVG(salary) AS avg_sal
FROM    employees
WHERE   hire_date > '2020-01-01'  -- filters rows BEFORE GROUP BY
GROUP BY dept
HAVING  AVG(salary) > 50000;      -- filters groups AFTER GROUP BY
```

Q **OLTP vs OLAP differences**

FEATURE	OLTP	OLAP
Purpose	Transaction processing	Analytical queries
Data volume	Small, row-level	Large, aggregated
Operations	INSERT/UPDATE/DELETE	SELECT with heavy aggregation
Normalization	Highly normalized (3NF+)	Denormalized (star/snowflake schema)
Response time	Milliseconds	Seconds to minutes
Example	Order management, banking	Sales dashboards, reporting
GCP tool	Cloud SQL, Spanner	BigQuery

Q **JOIN types — output analysis (Table A: 6 records, Table B: 7 records)**

Given: Table A IDs: 1,2,0 (6 records total). Table B IDs: 1,1,1,2,2,0,0 (7 records).

JOIN TYPE	EXPECTED OUTPUT
INNER JOIN	Matching rows: $1 \times 3 + 2 \times 2 + 0 \times 2 = 7$ rows
LEFT JOIN	All 6 from A + matching from B (NULLs for A rows with no match)

RIGHT JOIN	All 7 from B + matching from A
FULL OUTER JOIN	All unique combinations; NULLs where no match
CROSS JOIN	$6 \times 7 = 42$ rows (Cartesian product)
LEFT ANTI JOIN	Rows in A with NO match in B

Q Customers who bought A and B but NOT C



●SQL — Conditional Product Purchase Filter

```
SELECT customer_id
FROM   orders
WHERE  product IN ('A', 'B', 'C')
GROUP BY customer_id
HAVING
    SUM(CASE WHEN product = 'A' THEN 1 ELSE 0 END) > 0
AND SUM(CASE WHEN product = 'B' THEN 1 ELSE 0 END) > 0
AND SUM(CASE WHEN product = 'C' THEN 1 ELSE 0 END) = 0;
```

Q Customers who bought in each department



●SQL — Customers in All Departments

```
SELECT customer_id
FROM   orders o
JOIN   products p ON o.product_id = p.product_id
GROUP BY customer_id
HAVING COUNT(DISTINCT p.department) =
    (SELECT COUNT(DISTINCT department) FROM products);
```

Q Third least amount — display all records with same amount



●SQL — 3rd Least Amount (ties included)

```
SELECT *
FROM (
    SELECT *,
           DENSE_RANK() OVER (ORDER BY amount ASC) AS rn
    FROM orders
) t
WHERE rn = 3;
```

Note: DENSE_RANK ensures all records tied at the 3rd least amount are returned.

Q Gender-wise count per department (pivot style)



●SQL — Gender Count by Department

```
-- Input: stud_id, name, gender, dept_nm
SELECT
  dept_nm,
  SUM(CASE WHEN gender = 'M' THEN 1 ELSE 0 END) AS male_cnt,
  SUM(CASE WHEN gender = 'F' THEN 1 ELSE 0 END) AS female_cnt
FROM   students
GROUP BY dept_nm;
```

Q CTE vs Temp Table — what is the difference?

FEATURE	CTE (WITH CLAUSE)	TEMP TABLE
Scope	Single query only	Session scope (multiple queries)
Performance	May re-execute	Materialized — executes once
Indexing	Not possible	Possible
Storage	No disk storage	Stored in tempdb/session
Use case	Readability, recursive queries	Reusable intermediate results

Q Stored Procedure vs User Defined Function (UDF)

FEATURE	STORED PROCEDURE	UDF
Return value	Optional / multiple result sets	Must return a value
DML in body	Yes (INSERT/UPDATE/DELETE)	No (SELECT only in SQL UDFs)
Call from SELECT	No	Yes
Transaction control	Yes (BEGIN/COMMIT)	No
Use case	Complex business logic, ETL steps	Reusable expression in queries

Note: You CAN call a UDF inside a stored procedure. You CANNOT call a stored procedure inside a UDF.

Q Primary Key vs Unique Key

- **Primary Key:** Uniquely identifies each row; cannot be NULL; only ONE per table.
- **Unique Key:** Ensures uniqueness; CAN be NULL (once); multiple allowed per table.

Note in BigQuery: Both are informational constraints only (not enforced) — used for optimization hints.

Q UNION vs UNION ALL

- **UNION** — Combines results and removes duplicates (performs a DISTINCT operation). Slower.
- **UNION ALL** — Combines results keeping all rows including duplicates. Faster — no dedup step. Preferred in BigQuery for performance.

Q Can TRUNCATE and DELETE be rolled back?

- **DELETE:** DML — can be rolled back within a transaction.
- **TRUNCATE:** DDL in most RDBMS — cannot be rolled back (auto-committed).

In BigQuery: Both can be undone within the 7-day time travel window using `FOR SYSTEM_TIME AS OF`.

Q What is normalization and its normal forms?

Normalization organizes data in a relational DB to eliminate redundancy and improve data integrity.

NORMAL FORM	RULE
1NF	Atomic values; no repeating groups; each row unique
2NF	1NF + no partial dependency on composite primary key
3NF	2NF + no transitive dependencies between non-key columns
BCNF	Every determinant is a candidate key (stronger 3NF)

Q OLTP vs OLAP — differences

FEATURE	OLTP	OLAP
Purpose	Transaction processing	Analytical queries
Data volume	Small, row-level	Large, aggregated
Operations	INSERT / UPDATE / DELETE	SELECT with heavy aggregation
Normalization	Highly normalized (3NF+)	Denormalized (star/snowflake schema)
Response time	Milliseconds	Seconds to minutes
Example	Order management, banking	Sales dashboards, reporting
GCP tool	Cloud SQL, Spanner	BigQuery

Q Types of JOINS — explain each with output analysis

Given: **Table A** — 6 records, IDs: 1,1,2,2,0,0 | **Table B** — 7 records, IDs: 1,1,1,2,2,0,0

JOIN TYPE	OUTPUT ROWS	DESCRIPTION
INNER JOIN	$1 \times 3 + 2 \times 2 + 0 \times 2 = \mathbf{7 \text{ rows}}$	Only matching rows from both tables
LEFT JOIN	All 6 from A + matching from B	All A rows; NULL for unmatched B side
RIGHT JOIN	All 7 from B + matching from A	All B rows; NULL for unmatched A side
FULL OUTER JOIN	All combinations; NULLs where no match	All rows from both sides
CROSS JOIN	$6 \times 7 = \mathbf{42 \text{ rows}}$	Cartesian product — every A row with every B row
LEFT ANTI JOIN	Rows in A with NO match in B	Use: <code>WHERE B.key IS NULL</code> after LEFT JOIN

SQL — JOIN Examples

```
-- INNER JOIN: only matched rows
SELECT a.*, b.sal
FROM TableA a INNER JOIN TableB b ON a.id = b.id;

-- LEFT JOIN: all from A, NULL if no match in B
SELECT a.emp_id, a.name, b.sal
FROM Table1 a LEFT JOIN Table2 b ON a.emp_id = b.emp_id;

-- FULL OUTER JOIN
SELECT COALESCE(a.id, b.id) AS id, a.name, b.sal
FROM TableA a FULL OUTER JOIN TableB b ON a.id = b.id;

-- CROSS JOIN: Cartesian product
SELECT a.name, b.product
FROM customers a CROSS JOIN products b;
```

Q Find 3rd highest salary

SQL — 3rd Highest Salary

```
-- Using DENSE_RANK
SELECT emp_name, salary
FROM (
    SELECT emp_name, salary,
           DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
)
WHERE rnk = 3;

-- Using LIMIT / OFFSET
SELECT DISTINCT salary
FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 2;
```

Q Customers who bought A and B but NOT C



SQL — Conditional Product Filter

```
SELECT customer_id
FROM orders
WHERE product IN ('A', 'B', 'C')
GROUP BY customer_id
HAVING
    SUM(CASE WHEN product = 'A' THEN 1 ELSE 0 END) > 0 -- bought A
AND SUM(CASE WHEN product = 'B' THEN 1 ELSE 0 END) > 0 -- bought B
AND SUM(CASE WHEN product = 'C' THEN 1 ELSE 0 END) = 0; -- never bought C
```

Q Customers who bought in each department



SQL — All Departments Coverage

```
SELECT o.customer_id
FROM orders o
JOIN products p ON o.product_id = p.product_id
GROUP BY o.customer_id
HAVING COUNT(DISTINCT p.department) = (
    SELECT COUNT(DISTINCT department) FROM products
);
```

Q Composite Primary Key — can a table have two primary keys?

A table can have only **one primary key**, but that key can be **composite** — made up of two or more columns together.



SQL — Composite Primary Key & Duplicate Detection

```
-- Composite PK: student_id + subject_id together must be unique
CREATE TABLE enrollments (
    student_id INT64,
    subject_id INT64,
    marks      FLOAT64,
    PRIMARY KEY (student_id, subject_id) NOT ENFORCED
);

-- Find duplicates on composite key
SELECT student_id, subject_id, COUNT(*) AS cnt
FROM enrollments
GROUP BY student_id, subject_id
HAVING COUNT(*) > 1;
```

Q Pattern matching using LIKE



SQL — LIKE Pattern Matching

```
-- % matches zero or more characters
SELECT * FROM employees WHERE name LIKE 'A%';      -- starts with A
SELECT * FROM employees WHERE name LIKE '%kumar';  -- ends with kumar
SELECT * FROM employees WHERE name LIKE '%an%';    -- contains 'an'

-- _ matches exactly one character
SELECT * FROM employees WHERE name LIKE '_ohn';    -- John, Bohn, etc.

-- BigQuery: use LIKE or REGEXP_CONTAINS for complex patterns
SELECT * FROM employees
WHERE REGEXP_CONTAINS(email, r'@gmail\.com$');
```

Q **OLTP vs OLAP differences**

FEATURE	OLTP	OLAP
Purpose	Transaction processing	Analytical queries
Data volume	Small, row-level	Large, aggregated
Operations	INSERT / UPDATE / DELETE	SELECT with heavy aggregation
Normalization	Highly normalized (3NF+)	Denormalized (star/snowflake schema)
Response time	Milliseconds	Seconds to minutes
Example	Order management, banking	Sales dashboards, reporting
GCP tool	Cloud SQL, Spanner	BigQuery

Q **JOIN output analysis — Table A (6 records) vs Table B (7 records)**

Given: Table A IDs: 1, 1, 2, 2, 0, 0 (6 records). Table B IDs: 1, 1, 1, 2, 2, 0, 0 (7 records).

JOIN TYPE	EXPECTED OUTPUT
INNER JOIN	Matching rows only: $1 \times 3 + 2 \times 2 + 0 \times 2 =$ matching cross product (e.g. 2 A-rows of 1×3 B-rows of $1 = 6$, plus $2 \times 2 = 4$, plus $2 \times 2 = 4 \rightarrow 14$ rows)
LEFT JOIN	All 6 rows from A + matching from B (NULLs for A rows with no match in B)
RIGHT JOIN	All 7 rows from B + matching from A
FULL OUTER JOIN	All rows from both sides; NULLs where no match
CROSS JOIN	$6 \times 7 = 42$ rows (Cartesian product)
LEFT ANTI JOIN	Rows in A with NO match in B (0 rows if all A IDs exist in B)

Key rule: When both sides have duplicates on the join key, the result is a Cartesian product *per key group* — n rows from A $\times$ m rows from B for each matching key value.

Q Composite Primary Key — can a table have two primary keys?

A table can have only **one primary key**, but that primary key can be **composite** — made up of two or more columns together. The combination must be unique across rows.



SQL — Composite Primary Key & Find Duplicates

```
-- Define composite primary key
CREATE TABLE order_items (
  order_id    INT64,
  product_id  INT64,
  quantity    INT64,
  PRIMARY KEY (order_id, product_id) NOT ENFORCED
);

-- Find duplicates on composite key
SELECT  order_id, product_id, COUNT(*) AS cnt
FROM    order_items
GROUP BY order_id, product_id
HAVING  COUNT(*) > 1;
```

BigQuery note: PRIMARY KEY constraints are *informational only* (NOT ENFORCED) — used as optimization hints, not actually enforced.

Q Partition in SQL — can I partition 20 columns / 10,000 rows?

In BigQuery, a table supports **exactly one partition column**. You cannot partition on 10 columns simultaneously. The supported partition column types are `DATE`, `TIMESTAMP`, `DATETIME`, or `INT64` (range).

- For **20 columns / 10,000 rows**: partitioning gives little benefit at this scale — BQ full scans are fast. Consider **clustering** (up to 4 columns) instead.
- If no DATE/TIMESTAMP column exists, use **ingestion-time partitioning** (`_PARTITIONTIME`).
- To filter across multiple columns efficiently, combine 1 partition column + up to 4 cluster columns.



SQL — Partition + Cluster (Best Practice)

```
-- One partition column + up to 4 cluster columns
CREATE TABLE ds.emp
PARTITION BY DATE(hire_date)
CLUSTER BY dept, location, salary_band, status
AS SELECT * FROM raw.emp;
```

Q Normalization — 1NF, 2NF, 3NF, BCNF

Normalization organizes data in a relational DB to eliminate redundancy and improve data integrity.

NORMAL FORM	RULE
1NF	Atomic values; no repeating groups; each row unique
2NF	1NF + no partial dependency on composite primary key

3NF	2NF + no transitive dependencies between non-key columns
BCNF	Every determinant is a candidate key (stronger 3NF)

OLAP / BigQuery: Normalization is deliberately *avoided* in analytics warehouses — denormalized star schemas with STRUCTs/ARRAYs perform better than normalized joins.

Q CTE vs Temp Table — differences

FEATURE	CTE (WITH CLAUSE)	TEMP TABLE
Scope	Single query only	Session scope (multiple queries)
Performance	May re-execute per reference	Materialized — executes once
Indexing	Not possible	Possible
Storage	No disk storage	Stored in tempdb/session
Use case	Readability, recursive queries	Reusable intermediate results

Q Stored Procedure vs User Defined Function (UDF)

FEATURE	STORED PROCEDURE	UDF
Return value	Optional / multiple result sets	Must return a value
DML in body	Yes (INSERT / UPDATE / DELETE)	No (SELECT only in SQL UDFs)
Call from SELECT	No	Yes
Transaction control	Yes (BEGIN / COMMIT)	No
Use case	Complex business logic, ETL steps	Reusable expression in queries

Rule: You CAN call a UDF inside a stored procedure. You CANNOT call a stored procedure inside a UDF.

Q UNION vs UNION ALL

- **UNION** — Combines results and removes duplicates (performs a DISTINCT operation). Slower.
- **UNION ALL** — Combines all rows including duplicates. Faster — no dedup step. **Preferred in BigQuery** for performance.

Q Can TRUNCATE and DELETE be rolled back?

- **DELETE** — DML statement; can be rolled back within an open transaction.
- **TRUNCATE** — DDL in most RDBMS; auto-committed and cannot be rolled back.

- In BigQuery — Both can be undone within the **7-day time travel** window using `FOR SYSTEM_TIME AS OF`.

Q Primary Key vs Unique Key

- **Primary Key** — Uniquely identifies each row; *cannot be NULL*; only **one** per table.
- **Unique Key** — Ensures uniqueness; *can be NULL* (once); multiple allowed per table.

BigQuery note: Both are informational constraints only (NOT ENFORCED) — used as query optimization hints.

Q Customers who bought A and B but NOT C

```
SELECT  customer_id
FROM    orders
WHERE   product IN ('A', 'B', 'C')
GROUP BY customer_id
HAVING  SUM(CASE WHEN product = 'A' THEN 1 ELSE 0 END) > 0
AND SUM(CASE WHEN product = 'B' THEN 1 ELSE 0 END) > 0
AND SUM(CASE WHEN product = 'C' THEN 1 ELSE 0 END) = 0;
```

Q Customers who bought in each department

```
SELECT customer_id
FROM   orders o
JOIN   products p ON o.product_id = p.product_id
GROUP BY customer_id
HAVING COUNT(DISTINCT p.department) = (
    SELECT COUNT(DISTINCT department) FROM products
);
```

Q Third least amount — display all records with same amount

```
SELECT *
FROM (
    SELECT o.*,
           DENSE_RANK() OVER (ORDER BY amount ASC) AS rn
    FROM   orders o
) t
WHERE rn = 3;
```

```
-- DENSE_RANK ensures all rows with the 3rd-lowest amount are returned,  
-- even if multiple rows share the same amount value.
```

Q BigQuery — fetch STRUCT data from an array where attribute_id = 182

When you have an array of STRUCTs (e.g. an `addendum` array), use `UNNEST` to flatten it, then filter by the struct field.

●SQL — UNNEST Array, Filter by attribute_id

```
-- Table schema: records (id INT64, addendum ARRAY<STRUCT<attribute_id INT64, value STRING>>)
```

```
SELECT  
  r.id,  
  a.attribute_id,  
  a.value  
FROM   records r,  
UNNEST(r.addendum) AS a  
WHERE  a.attribute_id = 182;
```

```
-- Alternative: keep the outer row context using LEFT JOIN UNNEST  
-- (returns NULL for records that have no matching addendum element)
```

```
SELECT  
  r.id,  
  a.attribute_id,  
  a.value  
FROM   records r  
LEFT JOIN UNNEST(r.addendum) AS a  
  ON a.attribute_id = 182;
```

Q Query optimization in SQL — query running 1 hour instead of 10 minutes

- **Check Execution Plan** — use `EXPLAIN` or BQ Job Information to find the slow step.
- **Check data volume** — did source data grow significantly? Is a partition filter missing?
- **Check for full table scan** — is the partition column used in the WHERE clause?
- **Check for skew** — is one task/slot doing 80% of the work?
- **Check indexes** — have indexes been dropped or fragmented?
- **Check concurrent load** — was the system under heavy load causing slot contention?
- **Check for new joins** — were Cartesian products introduced by schema changes?
- **Rewrite query** — replace subqueries with CTEs, add filters, use approximate functions.
- **Use indexes on filter/join columns**; avoid functions on indexed columns in WHERE.

Q Find duplicates using GROUP BY and ROW_NUMBER

●SQL — Identify Duplicates

```
-- Method 1: GROUP BY — find duplicate emp_ids  
SELECT  emp_id, COUNT(*) AS cnt  
FROM    employees  
GROUP BY emp_id
```

```

HAVING     COUNT(*) > 1;

-- Method 2: ROW_NUMBER — flag all duplicates
SELECT *
FROM (
    SELECT *,
           ROW_NUMBER() OVER (
               PARTITION BY emp_id
               ORDER BY updated_at DESC
           ) AS rn
    FROM employees
)
WHERE rn > 1;  -- rn=1 is the "keep" row; rn>1 are duplicates

```

CHAPTER 04 Airflow / Cloud Composer

Q What are the different Airflow operators?

OPERATOR	PURPOSE
PythonOperator	Execute a Python callable
BashOperator	Run bash shell commands
BigQueryOperator	Execute BigQuery SQL
GCSToBigQueryOperator	Load data from GCS to BQ
BigQueryToGCSOperator	Export BQ table to GCS
DataflowTemplatedJobStartOperator	Launch a Dataflow template
DataprocSubmitJobOperator	Submit Spark job to Dataproc
BranchPythonOperator	Conditional branching in DAG
GCSObjectExistenceSensor	Wait for file to appear in GCS
ExternalTaskSensor	Wait for task in another DAG

Q T1 runs first, then T2 and T3 run in parallel — how?

```

Python — Parallel Task DAG

from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime

with DAG(
    'parallel_dag',
    start_date=datetime(2024, 1, 1),
    schedule_interval='@daily'
) as dag:

```

```

t1 = PythonOperator(task_id='task1', python_callable=task1_fn)
t2 = PythonOperator(task_id='task2', python_callable=task2_fn)
t3 = PythonOperator(task_id='task3', python_callable=task3_fn)

t1 >> [t2, t3] # t2 and t3 run in parallel after t1

```

Q T1 always runs, but either T2 OR T3 should run — Branching

Python — BranchPythonOperator

```

from airflow.operators.python import BranchPythonOperator
from airflow.operators.empty import EmptyOperator

def choose_branch(**context):
    if some_condition:
        return 'task2'
    else:
        return 'task3'

t1 = PythonOperator(task_id='task1', python_callable=task1_fn)
branch = BranchPythonOperator(task_id='branch', python_callable=choose_branch)
t2 = PythonOperator(task_id='task2', python_callable=task2_fn)
t3 = PythonOperator(task_id='task3', python_callable=task3_fn)
end = EmptyOperator(task_id='end',
                    trigger_rule='none_failed_min_one_success')

t1 >> branch >> [t2, t3] >> end

```

Q How do tasks communicate in Airflow? (XCom)

XCom (Cross-Communication): Key-value store in Airflow's metadata DB for small data exchange between tasks.

Python — XCom Push & Pull

```

# Push XCom value from a task
def push_fn(**context):
    context['ti'].xcom_push(key='result', value='some_data')

# Pull XCom value in another task
def pull_fn(**context):
    val = context['ti'].xcom_pull(
        task_ids='push_task',
        key='result'
    )
    print(val)

```

Note: XCom is for small data only (< a few KB). For large data, write to GCS and pass the GCS path via XCom.

Q Dynamic DAG — migrate 10 tables from GCS to BQ



```
from airflow import DAG
from airflow.providers.google.cloud.transfers.gcs_to_bigquery import GCSToBigQueryOperator
from datetime import datetime

tables = [
    'orders', 'customers', 'products', 'sales',
    'inventory', 'returns', 'shipping', 'payments',
    'vendors', 'employees'
]

with DAG(
    'dynamic_gcs_to_bq',
    start_date=datetime(2024, 1, 1),
    schedule_interval='@daily',
    catchup=False
) as dag:

    for table in tables:
        GCSToBigQueryOperator(
            task_id                = f'load_{table}',
            bucket                  = 'my-bucket',
            source_objects          = [f'data/{table}/*.csv'],
            destination_project_dataset_table= f'my_project.dataset.{table}',
            source_format           = 'CSV',
            write_disposition       = 'WRITE_TRUNCATE',
            skip_leading_rows       = 1,
            autodetect              = True,
        )
```

Q What are Airflow Connections?

Airflow Connections store credentials for external systems (BigQuery, GCS, databases, APIs). Defined in Airflow UI under **Admin → Connections** and referenced by `conn_id` in operators.



```
from airflow.providers.google.cloud.operators.bigquery import BigQueryInsertJobOperator

run_query = BigQueryInsertJobOperator(
    task_id    = 'run_bq_query',
    gcp_conn_id= 'google_cloud_default', # conn_id defined in Admin → Connections
    configuration={
        'query': {
            'query'          : 'SELECT COUNT(*) FROM `project.dataset.table`',
            'useLegacySql'   : False,
        }
    }
)
```

Connections can also be stored as **environment variables** (`AIRFLOW_CONN_<CONN_ID>`) or in **Secret Manager** for production security.

Q How to handle a bad or missing file in Airflow?

Use `GCSObjectExistenceSensor` to wait for the file before processing. If it never arrives, raise an alert and skip or fail gracefully.



Python — GCS File Sensor + Error Handling

```
from airflow.providers.google.cloud.sensors.gcs import GCSObjectExistenceSensor
from airflow.operators.python import PythonOperator

# Wait up to 1 hour for file; check every 60 seconds
check_file = GCSObjectExistenceSensor(
    task_id      = 'check_file_exists',
    bucket       = 'my-bucket',
    object       = 'data/input.csv',
    timeout      = 3600,
    poke_interval = 60,
    mode        = 'poke',
)

def handle_missing(**context):
    # Log alert, send Slack/email, or raise to fail the DAG
    raise FileNotFoundError('Expected file not found in GCS after timeout')

# Chain: check → process; sensor times out → on_failure_callback fires
check_file >> process_file
```

Best practice: Use `on_failure_callback` on the DAG to send Slack/email alerts when any task including the sensor fails.

CHAPTER 05 PySpark — Key Concepts & Code

Q How to create a SparkSession?



Python — SparkSession Creation

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName('MyApp') \
    .config('spark.executor.memory', '4g') \
    .config('spark.sql.shuffle.partitions', '200') \
    .getOrCreate()
```

Q Read CSV in PySpark and load to BigQuery



Python — CSV → Transform → BigQuery

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, trim, upper, when

spark = SparkSession.builder.appName('CSV_to_BQ').getOrCreate()
```



```

# Read CSV from GCS
df = spark.read.csv(
    'gs://my-bucket/data/employees.csv',
    header=True,
    inferSchema=True
)

# Apply transformations
df_clean = df \
    .filter(col('salary') > 0) \
    .withColumn('name', trim(upper(col('name')))) \
    .withColumn('dept', when(col('dept').isNull(), 'UNKNOWN').otherwise(col('dept'))) \
    .dropDuplicates(['emp_id'])

# Write to BigQuery
df_clean.write \
    .format('bigquery') \
    .option('table', 'project.dataset.employees') \
    .option('temporaryGcsBucket', 'my-temp-bucket') \
    .mode('overwrite') \
    .save()

```

Q What is data skew and salting technique?

Data Skew occurs when data is unevenly distributed across partitions — some tasks process far more data than others, causing bottlenecks.

Python — Salting to Fix Skewed Join

```

from pyspark.sql.functions import rand, concat, col, floor, explode, array, lit, broadcast

# Add salt to large (skewed) table
df_large = df_large \
    .withColumn('salt', (rand() * 10).cast('int')) \
    .withColumn('salted_key', concat(col('customer_id'), col('salt').cast('string')))

# Explode salt in small (lookup) table
df_small = df_small \
    .withColumn('salt', explode(array([lit(i) for i in range(10)]))) \
    .withColumn('salted_key', concat(col('customer_id'), col('salt').cast('string')))

# Join on salted key
df_result = df_large.join(broadcast(df_small), 'salted_key')

```

Q How to handle streaming data from Pub/Sub to BigQuery?

Python — Apache Beam (Dataflow) Streaming

```

import apache_beam as beam
from apache_beam.options.pipeline_options import PipelineOptions

pipeline = beam.Pipeline(options=PipelineOptions(runner='DataflowRunner'))

```

```
(
    pipeline
    | 'ReadFromPubSub' >> beam.io.ReadFromPubSub(
        topic='projects/proj/topics/topic')
    | 'ParseJSON' >> beam.Map(lambda x: json.loads(x))
    | 'WriteToBQ' >> beam.io.WriteToBigQuery(
        'project:dataset.table',
        write_disposition=beam.io.BigQueryDisposition.WRITE_APPEND)
)
pipeline.run()
```



Python — PySpark Structured Streaming (Dataproc)

```
df_stream = spark.readStream \
    .format('pubsub') \
    .option('topic', 'projects/proj/topics/topic') \
    .load()

df_stream.writeStream \
    .format('bigquery') \
    .option('table', 'project.dataset.table') \
    .option('checkpointLocation', 'gs://bucket/checkpoints/') \
    .outputMode('append') \
    .start()
```

Q Find second largest element in a list without built-in functions



Python — Second Largest Without Builtins

```
def second_largest(lst):
    first = second = float('-inf')
    for num in lst:
        if num > first:
            second = first
            first = num
        elif num > second and num != first:
            second = num
    return second if second != float('-inf') else None

print(second_largest([3, 1, 4, 1, 5, 9, 2, 6])) # Output: 6
```

Logic: Single pass — track `first` (largest) and `second` (second largest). Update `second` only when the number is less than `first` but greater than current `second`.

CHAPTER 06 Python Fundamentals

Q Mutable vs Immutable data types in Python

TYPE

MUTABLE?

EXAMPLES

list	Yes	[1, 2, 3]
dict	Yes	{'a': 1}
set	Yes	{1, 2, 3}
str	No	'hello'
tuple	No	(1, 2, 3)
int / float	No	42, 3.14
frozenset	No	frozenset({1, 2})

Q Lambda, map(), and filter()

Python — Lambda / map / filter

```
# Lambda — anonymous single-expression function
add = lambda x, y: x + y
print(add(3, 4)) # 7

# map() — apply function to each element
doubled = list(map(lambda x: x * 2, [1, 2, 3])) # [2, 4, 6]

# filter() — keep elements matching condition
positives = list(filter(lambda x: x > 0, [-1, 2, -3, 4])) # [2, 4]

# sorted with lambda key
nums = [3, 1, 4, 1, 5]
sorted_desc = sorted(nums, key=lambda x: -x) # descending
```

Q Remove duplicates from list while preserving order

Python — Deduplication Methods

```
lst = [1, 2, 1, 3, 4, 5, 2, 3, 3]

# Method 1: dict.fromkeys (Python 3.7+ preserves insertion order)
result = list(dict.fromkeys(lst))

# Method 2: set with seen tracking
seen = set()
result = [x for x in lst if not (x in seen or seen.add(x))]

print(result) # [1, 2, 3, 4, 5]
```

Q Key Python libraries for Data Engineering

LIBRARY	USE
---------	-----

pandas	Data manipulation and analysis
pyspark	Distributed data processing
google-cloud-bigquery	BigQuery Python client
google-cloud-storage	GCS Python client
apache-beam	Dataflow pipeline SDK
apache-airflow	Workflow orchestration
pyarrow	Columnar data; Parquet I/O
great_expectations	Data quality validation
dbt-core	Data transformation framework

Q Deduplicate records from a list of dictionaries

Python — Deduplicate List of Dicts

```
data = [
    {'id': 1, 'name': 'Ravi'},
    {'id': 2, 'name': 'Anita'},
    {'id': 1, 'name': 'Ravi'},    # duplicate
    {'id': 3, 'name': 'Kumar'},
]

# Method 1: Manual loop check
unique = []
for item in data:
    if item not in unique:
        unique.append(item)

# Method 2: Use dict keyed by id (preserves last occurrence)
unique = list({d['id']: d for d in data}.values())

# Method 3: Using frozensets (if all values are hashable)
seen = set()
unique = []
for d in data:
    key = frozenset(d.items())
    if key not in seen:
        seen.add(key)
        unique.append(d)

print(unique)
# [{'id': 1, 'name': 'Ravi'}, {'id': 2, 'name': 'Anita'}, {'id': 3, 'name': 'Kumar'}]
```

Q What is flatten and flatMap in Python?

Flatten converts a nested list into a single flat list. **flatMap** is the combination of map + flatten — map a function then flatten the result.

```
import itertools

# Flatten nested list
nested = [[1, 2], [3, 4], [5]]
flat = list(itertools.chain.from_iterable(nested))
print(flat) # [1, 2, 3, 4, 5]

# Python equivalent of flatMap: map then flatten
sentences = ['Hello World', 'Spark is fast']
words = [w for sentence in sentences for w in sentence.split()]
print(words) # ['Hello', 'World', 'Spark', 'is', 'fast']

# filter() to remove 0 values
values = [0, 1, 0, 3, 4, 0]
filtered = [x for x in values if x != 0]
print(filtered) # [1, 3, 4]
```

In PySpark, `flatMap()` applies a function to each element and flattens the result — e.g., splitting sentences into individual words from an RDD.

Q How to hide personal info / Data Masking (PII handling)?

- **Tokenization:** Replace PII with a reversible token stored in a secure vault.
- **Encryption:** Encrypt PII columns at rest and in transit (CMEK in BigQuery).
- **Dynamic Data Masking (BigQuery):** Column-level policy tags — unauthorized users see NULL.
- **Pseudonymization:** Replace identifying fields with pseudonyms (fake but consistent IDs).
- **Data Hashing:** Hash PII values (SHA-256) for join keys without exposing real values.
- **Authorized Views:** Expose only non-sensitive columns to BI users.

```
import hashlib

def hash_pii(value: str) -> str:
    return hashlib.sha256(value.encode()).hexdigest()

# Use hashed value as join key — original PII never exposed
df['hashed_email'] = df['email'].apply(hash_pii)
df.drop(columns=['email'], inplace=True)
```

Q What is SOX compliance?

SOX (Sarbanes-Oxley Act) is a US law requiring accurate financial reporting and strong internal controls. In data engineering it means:

- **Audit trails:** Log all data access and modifications — who touched what data and when.
- **Access controls:** Role-based access; strict separation of duties.
- **Data integrity:** Ensure financial data is not altered without authorization.
- **Change management:** Document all pipeline changes with approvals.
- **Data retention:** Maintain financial data for legally required periods (typically 7 years).

- In **GCP**: Cloud Audit Logs, VPC Service Controls, CMEK, and IAM enforce SOX requirements.

CHAPTER 07 Data Modelling — Star, Snowflake, SCD, dbt

Q Fact vs Dimension Tables

ATTRIBUTE	FACT TABLE	DIMENSION TABLE
Content	Measures (sales_amount, qty, revenue)	Descriptive attrs (name, city, category)
Size	Very large (billions of rows)	Smaller (thousands-millions)
Keys	Foreign keys to all dimensions	Primary key
Examples	sales_fact, orders_fact	dim_customer, dim_product, dim_date
Grain	One row = one transaction/event	One row = one entity

Q What is dbt and its key functions?

dbt (data build tool) is a SQL-first transformation framework that runs inside your data warehouse.

FUNCTION	PURPOSE
ref('model_name')	Reference another dbt model — builds dependency graph
source('schema', 'table')	Reference raw source table
config()	Set materialization, partition, cluster settings
var('variable')	Access variables passed at runtime
is_incremental()	Check if model runs in incremental mode
target.name	Get current environment (dev/prod)

Q dbt incremental model

●●●

SQL (dbt) — Incremental Model

```
-- models/my_model.sql
{{ config(
    materialized = 'incremental',
    partition_by = {'field': 'event_date', 'data_type': 'date'},
    cluster_by   = ['user_id']
) }}

SELECT *
FROM {{ source('raw', 'events') }}

{% if is_incremental() %}
    WHERE event_date > (SELECT MAX(event_date) FROM {{ this }})
```

```
{% endif %}
```

Q SCD Types — Type 1 vs Type 2

TYPE	STRATEGY	KEEPS HISTORY?	USE CASE
SCD Type 1	Overwrite the old value — no history kept	No	Correcting errors (e.g. fixing a typo in a name)
SCD Type 2	Insert a new row with effective dates + is_current flag	Yes	Track customer address changes over time
SCD Type 3	Add a new column for previous value	Partial (1 version)	Only last change needed



● SQL — SCD Type 1 (Overwrite)

```
-- SCD Type 1: simply UPDATE — history is lost
MERGE dim_customer AS target
USING staging_customer AS source
  ON target.customer_id = source.customer_id
WHEN MATCHED THEN UPDATE SET
  target.email      = source.email,
  target.address    = source.address,
  target.updated_at = CURRENT_TIMESTAMP()
WHEN NOT MATCHED THEN INSERT
  (customer_id, email, address, updated_at)
VALUES
  (source.customer_id, source.email, source.address, CURRENT_TIMESTAMP());
```

Q Star Schema vs Snowflake Schema

FEATURE	STAR SCHEMA	SNOWFLAKE SCHEMA
Structure	Fact table + denormalized dimensions	Fact table + normalized dimensions
Joins	Fewer joins (1 level)	More joins (multiple levels)
Query performance	Faster (fewer joins)	Slower (more joins)
Storage	More storage (redundancy)	Less storage (normalized)
Complexity	Simpler	More complex
BQ recommendation	Preferred (denormalized)	Less preferred

Q Data skew in BigQuery — identification and handling

Data Skew in BigQuery means partition or slot imbalance — some tasks/slots process far more data than others.

- **Identify:** Check BigQuery execution graph — look for uneven slot usage.

- **Partition pruning:** Ensure queries filter on partition column to avoid full scans.
- **Approximate aggregation:** Use `APPROX_COUNT_DISTINCT` instead of exact `COUNT(DISTINCT)`.
- **Pre-aggregate:** Aggregate before joining large tables.
- **Avoid high-cardinality GROUP BY** on non-clustered columns.
- **Is it possible to partition on a product column?** — Only if the column is `INT64` (use `RANGE_BUCKET`) or `DATE` / `TIMESTAMP`. String columns are not directly supported as partition keys.

Q Agile / Sprint methodology in data engineering

- **Agile:** Iterative development with short delivery cycles (sprints).
- **Sprint:** 1-2 week iteration delivering a shippable increment.
- **Story Points:** Relative complexity estimate (Fibonacci: 1, 2, 3, 5, 8, 13). Team agrees via Planning Poker.
- **Daily standup:** What did I do? What will I do? Any blockers?
- **Sprint review:** Demo completed work to stakeholders.
- **Retrospective:** What went well? What to improve?

Q Layered Architecture — Staging → Curated → Final

LAYER	ALSO KNOWN AS	DESCRIPTION
Raw/Landing	—	Data as-is from source (GCS bucket). No transformation.
Staging	Bronze	Lightly cleaned — type casting, null handling, deduplication.
Curated	Silver	Business logic — joins, aggregations, SCD handling.
Final	Gold / Serving	Analytics-ready — star schema, materialized views, BI-optimized.

CHAPTER 08 Data Extraction, Sources & Ingestion

Q How to extract data from Oracle and upload to GCS?

Python — Oracle → GCS

```
import cx_Oracle
from google.cloud import storage
import pandas as pd
import io

# Connect to Oracle and fetch data
conn = cx_Oracle.connect('username/password@host:1521/ORCL')
df = pd.read_sql('SELECT * FROM employees', con=conn)
conn.close()

# Upload to GCS as CSV (no local file)
client = storage.Client()
bucket = client.bucket('my-bucket')
blob = bucket.blob('data/employees.csv')
csv_buffer = io.StringIO()
df.to_csv(csv_buffer, index=False)
```



```
blob.upload_from_string(
    csv_buffer.getvalue(),
    content_type='text/csv'
)
print('Uploaded successfully')
```

Q How to load data from GCS to BigQuery using Python?

```
from google.cloud import bigquery

client = bigquery.Client(project='my-project')
table_id = 'my-project.dataset.table_name'

job_config = bigquery.LoadJobConfig(
    source_format = bigquery.SourceFormat.CSV,
    skip_leading_rows = 1,
    autodetect = True,
    write_disposition = bigquery.WriteDisposition.WRITE_APPEND
)

load_job = client.load_table_from_uri(
    'gs://my-bucket/data/employees.csv',
    table_id,
    job_config=job_config
)

load_job.result() # Wait for job to complete
print(f'Loaded {client.get_table(table_id).num_rows} rows')
```

Python - GCS to BigQuery

Q API integration with Secret Manager

```
import requests
from google.cloud import secretmanager

def get_secret(secret_id):
    client = secretmanager.SecretManagerServiceClient()
    name = f'projects/my-project/secrets/{secret_id}/versions/latest'
    response = client.access_secret_version(request={'name': name})
    return response.payload.data.decode('UTF-8')

api_key = get_secret('my-api-key')
response = requests.get(
    'https://api.example.com/data',
    headers = {'Authorization': f'Bearer {api_key}'},
    params = {'start_date': '2024-01-01'}
)
data = response.json()
```

Python - Secret Manager + REST API

Q How to handle schema changes in BigQuery using Python?

Python — Add Column to BQ Schema

```
from google.cloud import bigquery

client = bigquery.Client()
table_ref = client.get_table('project.dataset.table')

# Append new field to existing schema
new_schema = table_ref.schema + [
    bigquery.SchemaField('new_column', 'STRING', mode='NULLABLE')
]
table_ref.schema = new_schema
client.update_table(table_ref, ['schema'])
print('Schema updated successfully')
```

CHAPTER 09 Unix Commands, Cost Optimization & Misc

Q Key Unix commands for a Data Engineer

COMMAND	PURPOSE
pwd	Print current working directory
ls -la	List files with permissions
chmod 755 file.sh	Set file permissions (rwxr-xr-x)
grep 'pattern' file	Search text pattern in file
grep -r 'error' /logs/	Recursive search in directory
sed 's/old/new/g' file	Find and replace text
awk '{print \$1, \$3}' file	Process text by columns
head -20 / tail -50 file	View first/last N lines
wc -l file	Count lines in file
find / -name '*.csv'	Find files by pattern
df -h	Disk space usage
top / htop	Real-time system resource monitor
nohup command &	Run command in background

Q Cost optimization techniques in GCP / BigQuery

- Use **partitioned tables** — filter on partition column to avoid full scans.
- Use **clustering** — prune blocks to reduce bytes processed.
- **SELECT specific columns** — avoid `SELECT *`.

- Use **Materialized Views** to cache frequent aggregation queries.
- Set **table expiration** — auto-delete temp/staging tables.
- Use **BI Engine** — caches data in memory for fast dashboard queries.
- Use **preemptible/spot VMs** in Dataproc for cost savings.
- Use **appropriate GCS storage class** — Nearline/Coldline for infrequent data.
- Delete unused datasets, tables, and buckets.

Q GCS storage class lifecycle rules

Python — GCS Lifecycle Policy

```
lifecycle_config = {
    'rule': [
        {
            'action': {'type': 'SetStorageClass', 'storageClass': 'NEARLINE'},
            'condition': {'age': 30}          # move to Nearline after 30 days
        },
        {
            'action': {'type': 'SetStorageClass', 'storageClass': 'ARCHIVE'},
            'condition': {'age': 365}        # archive after 365 days
        },
        {
            'action': {'type': 'Delete'},
            'condition': {'age': 1825}       # delete after 5 years
        }
    ]
}
```

CHAPTER 10 Quick Reference — Cheat Sheet

Q BigQuery SQL Quick Reference

SQL — BigQuery Cheat Sheet

```
-- Partition + Cluster table
CREATE TABLE ds.sales
PARTITION BY DATE(sale_date)
CLUSTER BY region
AS SELECT ...;

-- Deduplicate with QUALIFY
SELECT * FROM table
QUALIFY ROW_NUMBER() OVER(PARTITION BY id ORDER BY updated_at DESC) = 1;

-- MERGE (upsert)
MERGE target T USING source S ON T.id = S.id
WHEN MATCHED THEN UPDATE SET T.col = S.col
WHEN NOT MATCHED THEN INSERT VALUES (S.id, S.col);

-- Time travel
SELECT * FROM table
FOR SYSTEM_TIME AS OF TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 1 HOUR);
```

```
-- Row-level security
CREATE ROW ACCESS POLICY fp ON ds.table
GRANT TO ('group:finance@co.com')
FILTER USING (dept = 'Finance');
```

PySpark Quick Reference

Python — PySpark Cheat Sheet

```
# SparkSession
spark = SparkSession.builder.appName('App').getOrCreate()

# Read data
df = spark.read.csv('gs://bucket/file.csv', header=True, inferSchema=True)
df = spark.read.parquet('gs://bucket/data/')
df = spark.read.json('gs://bucket/file.json')

# Transform
df = df \
    .filter(col('sal') > 0) \
    .withColumn('dept', upper(col('dept'))) \
    .dropDuplicates(['emp_id']) \
    .groupBy('dept').agg(avg('sal').alias('avg_sal'))

# Broadcast join (small table)
df_result = df_large.join(broadcast(df_small), 'id')

# Write to BigQuery
df.write \
    .format('bigquery') \
    .option('table', 'proj.ds.table') \
    .mode('overwrite') \
    .save()

# Cache frequently reused DataFrame
df.cache() # or df.persist(StorageLevel.MEMORY_AND_DISK)
```

Airflow DAG Quick Reference

Python — Airflow DAG Template

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.providers.google.cloud.transfers.gcs_to_bigquery import GCSToBigQueryOperator
from datetime import datetime, timedelta

with DAG(
    'my_dag',
    start_date = datetime(2024, 1, 1),
    schedule_interval = '@daily',
    catchup = False,
    default_args = {
```

```

        'retries':      2,
        'retry_delay':  timedelta(minutes=5)
    }
) as dag:

    t1 = PythonOperator(task_id='extract', python_callable=extract_fn)

    t2 = GCSToBigQueryOperator(
        task_id              = 'load',
        bucket                = 'my-bucket',
        source_objects        = ['data/*.csv'],
        destination_project_dataset_table= 'proj.ds.table',
        write_disposition     = 'WRITE_APPEND',
        autodetect            = True,
    )

    t1 >> t2

```

Day-to-Day Activities of a GCP Data Engineer

- Monitor Airflow/Composer DAGs — check failures, clear and rerun stuck tasks.
- Develop and maintain ETL pipelines (GCS → BQ, Oracle → GCS → BQ, Pub/Sub → Dataflow → BQ).
- Write and optimize SQL queries in BigQuery.
- Perform data quality checks on ingested data.
- Work with stakeholders to understand data requirements.
- Implement SCD, partitioning, and clustering strategies.
- Handle schema evolution and backward compatibility.
- Support Agile sprints — attend standups, update JIRA tickets.
- Code reviews and deployments via CI/CD (Cloud Build + GitHub).
- Monitor costs using BQ information schema and GCP Billing.

GCP Data Engineer Interview Guide

SUPPLEMENTARY Q&A — Missing Topics Added

SECTION A: PySpark — Missing Topics

Q: What is lazy evaluation and its uses?

Lazy Evaluation: Spark does **NOT** execute transformations immediately. It builds a logical DAG (plan) of all transformations and only runs them when an **action** is called (collect, count, show, write, save).

Why it matters:

- **Catalyst Optimiser:** Sees the full plan and reorders/merges operations (predicate pushdown, column pruning, join reordering).
- **Avoids unnecessary work:** If you filter after a map, Spark pushes the filter earlier automatically.
- **Pipeline fusion:** Combines multiple narrow transformations into a single stage — fewer passes over the data.
- **Fault tolerance:** The lineage graph lets Spark recompute only the lost partition without restarting the whole job.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col

spark = SparkSession.builder.appName("LazyDemo").getOrCreate()

# These lines build the logical plan – NO data is read yet
df = spark.read.csv("gs://bucket/large_file.csv", header=True, inferSchema=True)
df_filtered = df.filter(col("salary") > 50000)      # transformation – lazy
df_selected = df_filtered.select("emp_id", "name")  # transformation – lazy

# Action – plan is optimised THEN executed
df_selected.show(10)  # data is read and processed only now
```

Q: How do you read JSON in PySpark?

```
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType, IntegerType

spark = SparkSession.builder.appName("ReadJSON").getOrCreate()

# ■■ Single JSON file (one JSON object per line – default)
df = spark.read.json("gs://bucket/data/orders.json")

# ■■ Multiple files with wildcard
df = spark.read.json("gs://bucket/data/orders/*.json")

# ■■ Multi-line JSON (entire file is one JSON object)
df = (spark.read
      .option("multiline", True)
      .json("gs://bucket/data/config.json"))

# ■■ JSON with explicit schema (recommended for production)
schema = StructType([
    StructField("order_id", IntegerType(), True),
    StructField("customer_id", StringType(), True),
    StructField("amount", IntegerType(), True),
])
```

```
df = spark.read.schema(schema).json("gs://bucket/data/*.json")

df.printSchema()
df.show(5)
```

Q: Read a CSV file and select only the department column.

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("SelectDept").getOrCreate()

# Read CSV
df = spark.read.csv(
    "gs://bucket/employees.csv",
    header=True,
    inferSchema=True,
)

# Select only the department column
df_dept = df.select("department")
df_dept.show(10)

# Distinct departments
df_unique_depts = df.select("department").distinct()
df_unique_depts.show()
```

Q: What is broadcast join in PySpark? When to use it?

A **broadcast join** sends (broadcasts) a small DataFrame to every executor node so the large DataFrame does not need to be shuffled across the network. This avoids the expensive shuffle step in a standard sort-merge join.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import broadcast

spark = SparkSession.builder.appName("BroadcastJoin").getOrCreate()

df_orders = spark.read.parquet("gs://bucket/orders/") # large table
df_products = spark.read.parquet("gs://bucket/products/") # small lookup table

# Without broadcast – Spark shuffles both sides
df_slow = df_orders.join(df_products, "product_id")

# With broadcast – df_products is sent to all executors
df_fast = df_orders.join(broadcast(df_products), "product_id")

# Spark can also auto-broadcast when table < spark.sql.autoBroadcastJoinThreshold (default 10 MB)
df_fast.show()
```

- Use when one side is small (typically < 100–200 MB).
- Eliminates shuffle — significant speedup for skewed or large fact-table joins.
- Can be forced even if Spark's auto-threshold is not triggered.
- Do NOT broadcast if both tables are large — will cause driver OOM.

Q: How do you handle large datasets in Python?

- **Use PySpark:** Distribute processing across a cluster — best for TB+ data.
- **pandas chunking:** Read CSV in chunks with chunksize parameter to avoid OOM.
- **Dask:** Parallel pandas-compatible library for larger-than-memory data.
- **BigQuery Storage API:** Read BQ data efficiently via Arrow format (google-cloud-bigquery-storage).
- **Parquet + PyArrow:** Read only needed columns (column pruning) and filter rows at read time (predicate pushdown).
- **Generator functions:** Process one record at a time — zero extra memory.

```
import pandas as pd

# ■■ Option 1: Chunked reading
chunk_size = 100_000
for chunk in pd.read_csv("large_file.csv", chunksize=chunk_size):
    # process each chunk independently
    filtered = chunk[chunk["salary"] > 50000]
    # append to output / write to BQ

# ■■ Option 2: Generator for row-by-row processing
def read_rows(filepath):
    with open(filepath) as f:
        for line in f:
            yield line.strip().split(",")

for row in read_rows("data.csv"):
    process(row)    # one row at a time – minimal memory
```

SECTION B: Airflow — Missing Topics

Q: What is a Sensor in Airflow?

A **Sensor** is a special type of operator that keeps **poking** (checking) a condition repeatedly until it is True — then it lets the downstream tasks run. It is used to wait for external events before proceeding.

Sensor	What it waits for
GCSObjectExistenceSensor	A specific file to appear in a GCS bucket
BigQueryTableExistenceSensor	A BigQuery table to exist
ExternalTaskSensor	A task in another DAG to complete
HttpSensor	An HTTP endpoint to return a success status
S3KeySensor	A file to appear in an S3 bucket
DateTimeSensor	A specific date/time to be reached

```
from airflow.providers.google.cloud.sensors.gcs import GCSObjectExistenceSensor

# Wait up to 1 hour for the daily file to arrive
wait_for_file = GCSObjectExistenceSensor(
    task_id="wait_for_daily_orders",
    bucket="my-data-bucket",
    object="orders/{{ ds }}/orders.csv",    # templated with run date
    timeout=3600,                          # fail if file not found within 1 hour
    poke_interval=60,                      # check every 60 seconds
    mode="poke",                          # keeps worker slot; use "reschedule" to free slot
)

wait_for_file >> process_file    # runs only when file exists
```

★ Use `mode='reschedule'` in production so the sensor frees its worker slot between pokes — prevents slot starvation.

Q: A task has been failing for 7 days — how do you identify and resolve it?

- **Step 1 — Check Airflow logs:** Open the failing task in UI → Log tab → read full Python traceback to find the exact error.
- **Step 2 — Classify the error:** FileNotFoundError? ConnectionError? BigQuery quota exceeded? Timeout? Permission denied?
- **Step 3 — Check upstream tasks:** Did a sensor or dependency task fail, causing this task to be skipped or stuck?
- **Step 4 — Check data:** Is the source file missing? Is the staging table empty? Did source schema change?

- **Step 5 — Check catchup setting:** Was catchup=True creating a backlog of 7 days of runs all executing simultaneously and competing for resources?
- **Step 6 — Check permissions:** Was an IAM role removed from the service account?
- **Step 7 — Check data volume:** Did input data grow suddenly causing OOM or query timeout?
- **Step 8 — Reproduce locally:** Run the Python callable independently with the same arguments to isolate the bug.
- **Step 9 — Fix and clear:** After fixing, use Airflow UI → Clear failed task instances to trigger a fresh run.
- **Step 10 — Add alerting:** Set on_failure_callback with Slack/email so future failures alert immediately rather than after 7 days.

Q: Task1 success → Task2 runs; Task1 fail → Task3 runs. How?

Use **trigger_rule** parameter on the downstream tasks to control when they run based on the outcome of upstream tasks.

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime

with DAG("conditional_flow", start_date=datetime(2024, 1, 1),
        schedule_interval="@daily", catchup=False) as dag:

    def task1(): pass
    def task2(): print("Running because task1 succeeded")
    def task3(): print("Running because task1 failed")

    t1 = PythonOperator(task_id="task_1", python_callable=task1)

    # Runs ONLY if all upstream tasks succeeded (default behaviour)
    t2 = PythonOperator(
        task_id="task_2_on_success",
        python_callable=task2,
        trigger_rule="all_success",    # default – explicit here for clarity
    )

    # Runs ONLY if all upstream tasks failed
    t3 = PythonOperator(
        task_id="task_3_on_failure",
        python_callable=task3,
        trigger_rule="all_failed",
    )

    t1 >> t2    # t2 runs if t1 succeeds
    t1 >> t3    # t3 runs if t1 fails
```

trigger_rule	When the task runs
all_success (default)	All upstream tasks succeeded
all_failed	All upstream tasks failed
all_done	All upstream tasks finished (any state)
one_success	At least one upstream task succeeded
one_failed	At least one upstream task failed
none_failed	No upstream task failed (success or skipped)
none_failed_min_one_success	No failures AND at least one success

Q: Steps to create an Airflow DAG.

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.providers.google.cloud.transfers.gcs_to_bigquery import (
```

[illegible]

Q: What is catchup in Airflow?

catchup=True (Airflow default): When a DAG is enabled or its start_date is in the past, Airflow creates a DAG run for **every missed interval** between start date and today. Useful for historical backfills.

catchup=False: Airflow only runs the **most recent interval** — it does not backfill past missed runs. Recommended for production pipelines.

```
# BAD – start_date 1 year ago + catchup=True = 365 queued DAG runs
with DAG("bad_example",
        start_date=datetime(2024, 1, 1),
        schedule_interval="@daily",
        catchup=True) as dag:  # will trigger 365 backfill runs!
    ...

# GOOD – only runs the latest interval
with DAG("good_example",
        start_date=datetime(2024, 1, 1),
        schedule_interval="@daily",
        catchup=False) as dag:  # safe for production
```

★ If you need a one-time backfill, run: `airflow dags backfill -s 2024-01-01 -e 2024-01-31 my_dag_id`

Q: How to schedule DAG execution?

schedule_interval value	Meaning
'@daily'	Once a day at midnight (00:00 UTC)
'@hourly'	Once per hour at :00
'@weekly'	Once per week on Sunday midnight
'@monthly'	First day of each month at midnight
'0 6 * * *'	Cron — every day at 06:00 UTC
'0 6 * * 1-5'	Cron — 06:00 every weekday (Mon–Fri)
timedelta(hours=6)	Every 6 hours from start_date
None	Manual trigger only — no automatic schedule
'@once'	Run exactly once after start_date

Q: How to handle missing columns in a file using PythonOperator?

[illegible]

Q: How to run the same Python logic for multiple customers dynamically?

```
from airflow import DAG
from airflow.operators.python import PythonOperator
```

```
from datetime import datetime

CUSTOMERS = ["customer_alpha", "customer_beta", "customer_gamma"]

def process_customer(customer_id, **context):
    run_date = context["ds"]
    print(f"Processing data for {customer_id} on {run_date}")
    # load from GCS → transform → write to BQ partition for this customer

with DAG(
    dag_id="multi_customer_pipeline",
    start_date=datetime(2024, 1, 1),
    schedule_interval="@daily",
    catchup=False,
) as dag:

    # Dynamically create one task per customer
    for customer in CUSTOMERS:
        PythonOperator(
            task_id=f"process_{customer}",
            python_callable=process_customer,
            op_kwargs={"customer_id": customer},
        )

    # All customer tasks run in parallel (no dependency set between them)
```

SECTION C: BigQuery — Missing Topics

Q: What are BigQuery Slots and how many are allocated?

A **slot** is a virtual CPU unit used by BigQuery to execute query work. Complex queries use more slots; simple ones use fewer.

Tier	Slots	Billing
On-demand (default)	2,000 per project	Per TB scanned (\$5/TB; 1 TB/month free)
Standard commitment	100-slot blocks	1-year reservation — predictable cost
Enterprise	500-slot min	Cross-region failover; 1-year or 3-year
Flex Slots	Any amount	Short-term — buy slots for 60 seconds min

In interviews: For on-demand pricing say **2,000 slots per project**. For projects using reservations, the actual slot count depends on the committed tier purchased by the organisation.

Q: What are BigQuery pricing models?

Model	Description	Best for
On-demand	\$5 per TB scanned; 1 TB/month free	Ad-hoc, unpredictable workloads
Standard capacity	Purchase 100-slot blocks; 1-year commitment	Steady, predictable workloads
Enterprise	500-slot minimum; cross-region failover	Mission-critical pipelines
Enterprise Plus	3-year commitment; highest discount	Large-scale stable workloads
Flex Slots	Hourly/short-term slot purchase	Burst processing
Active storage	~\$0.02/GB/month	Tables actively changed
Long-term storage	~\$0.01/GB/month (auto after 90 days idle)	Historical/cold tables

Q: How to create a view in BigQuery?

[illegible]

to the GCS location.

Q: What are ETL tools? (list common ones)

Tool / Service	Type	Notes
Informatica PowerCenter	Enterprise ETL	Industry standard; complex transformations
Informatica IDMC	Cloud ETL / ELT	Modern cloud version of PowerCenter
Apache Beam / Dataflow	Managed pipeline	GCP-native; streaming + batch
Apache Spark / Dataproc	Distributed compute	Large-scale batch processing
Cloud Data Fusion	Visual ETL on GCP	Based on Apache CDAP; drag-and-drop
dbt	ELT / transformation	SQL-first; transforms inside the DW
Talend	Open-source ETL	GUI-based; Java-generated code
AWS Glue	Managed ETL	Spark-based; AWS ecosystem
Azure Data Factory	Cloud ETL	Azure-native pipeline tool
Fivetran / Airbyte	Data integration	Managed connectors for ingestion

Q: What is Informatica / PowerCenter? (ETL tool)

Informatica PowerCenter is an enterprise-grade ETL (Extract, Transform, Load) platform for data integration. It uses a graphical interface to design data pipelines without writing code.

Key components:

- **Repository Manager:** Stores all metadata (mappings, workflows, sessions).
- **Designer:** Where you build Source definitions, Target definitions, and Transformations (Aggregator, Joiner, Lookup, Filter, Expression, etc.).
- **Workflow Manager:** Schedules and runs sessions and workflows.
- **Workflow Monitor:** Monitor real-time status of running workflows.
- **Integration Service:** Executes the workflows on the server.

End-to-End pipeline using Informatica:

- **Step:** Connect Source (Oracle / flat file / DB) via Source Qualifier.
- **Step:** Apply transformations — Aggregator, Joiner, Lookup, Expression, Filter.
- **Step:** Map transformed data to Target definition (BQ / DW / flat file).
- **Step:** Create a Session using the Mapping.
- **Step:** Create a Workflow containing the Session.
- **Step:** Schedule the Workflow via Workflow Manager or external scheduler.

SECTION D: SQL — Missing Topics

Q: Students enrolled in subjects — JOIN query.

Given: students(stud_id, name), subjects(subject_id, subject_name), marks(stud_id, subject_id, marks)

```
-- Show each student, the subjects they enrolled in, and their marks
SELECT
    s.stud_id,
    s.name      AS student_name,
    sub.subject_name,
    m.marks
FROM
    students AS s
JOIN
    marks AS m
    ON s.stud_id = m.stud_id
```

```

JOIN
    subjects AS sub
    ON m.subject_id = sub.subject_id
ORDER BY
    s.stud_id, sub.subject_name;

-- Include students with no enrollments (LEFT JOIN)
SELECT
    s.stud_id,
    s.name AS student_name,
    COALESCE(sub.subject_name, 'None') AS subject_name,
    m.marks
FROM
    students AS s
LEFT JOIN marks AS m ON s.stud_id = m.stud_id
LEFT JOIN subjects AS sub ON m.subject_id = sub.subject_id;

```

Q: Customer who visits (buys) regularly — how to identify?

```

-- "Regularly" = purchased at least once every week for the last 4 weeks

WITH weekly_purchases AS (
    SELECT
        customer_id,
        DATE_TRUNC(order_date, WEEK) AS purchase_week
    FROM orders
    WHERE order_date >= DATE_SUB(CURRENT_DATE(), INTERVAL 28 DAY)
    GROUP BY customer_id, DATE_TRUNC(order_date, WEEK)
),
customer_week_count AS (
    SELECT
        customer_id,
        COUNT(DISTINCT purchase_week) AS weeks_active
    FROM weekly_purchases
    GROUP BY customer_id
)
-- Customers who bought in all 4 of the last 4 weeks
SELECT customer_id
FROM customer_week_count
WHERE weeks_active = 4;

-- Alternative: customers who bought on more than 3 distinct days last month
SELECT customer_id
FROM orders
WHERE order_date >= DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY)
GROUP BY customer_id
HAVING COUNT(DISTINCT DATE(order_date)) >= 3;

```

Q: Load data in pandas and remove blank header row.

```

import pandas as pd

# ■■■ Scenario: file has a completely blank first row before the actual header

# Option 1: Skip the blank row by specifying the header row index
df = pd.read_csv("data.csv", header=1) # row index 1 is the real header

# Option 2: Read normally then drop rows where all values are NaN
df = pd.read_csv("data.csv")
df = df.dropna(how="all") # drop rows where every column is NaN
df = df.reset_index(drop=True) # reset index after dropping

# Option 3: Strip blank column names (leading/trailing spaces)
df.columns = df.columns.str.strip()

# Option 4: Drop columns that have no name (Unnamed columns)
df = df.loc[:, ~df.columns.str.contains(r"^Unnamed")]

```

```
print(df.head())
print(f"Shape after cleaning: {df.shape}")
```

Q: count() vs count(3) in SQL — what is the difference?

```
-- count(*): counts ALL rows including rows with NULLs
SELECT COUNT(*) FROM employees;      -- counts every row

-- count(column): counts rows where that column is NOT NULL
SELECT COUNT(salary) FROM employees; -- skips rows where salary IS NULL

-- count(3): counts rows where the literal value 3 is not null
--          3 is never null, so this is equivalent to COUNT(*)
SELECT COUNT(3) FROM employees;      -- same result as COUNT(*)
```

Expression	What it counts	Includes NULLs?
COUNT(*)	Every row in the result set	Yes
COUNT(col)	Rows where col is NOT NULL	No
COUNT(3)	Every row (literal 3 is never NULL)	Yes (same as COUNT(*))
COUNT(DISTINCT col)	Distinct non-NULL values of col	No

Q: SQL Constraints — what are they?

Constraint	Purpose
PRIMARY KEY	Uniquely identifies each row; NOT NULL + UNIQUE; one per table
UNIQUE	Ensures all values in the column are distinct; allows one NULL
NOT NULL	Column cannot contain NULL values
FOREIGN KEY	Links to a primary key in another table; enforces referential integrity
CHECK	Ensures values satisfy a condition: CHECK (salary > 0)
DEFAULT	Provides a default value when none is supplied

★ In BigQuery, PRIMARY KEY and FOREIGN KEY are informational only (not enforced). They are hints to the query optimiser, not hard constraints.

Q: Partition in SQL — can I partition on 10 columns (20 cols, 10,000 rows)?

Partitioning is NOT about columns — it's about rows. You partition a table by **one column's values**, which divides the rows into separate segments. You cannot partition on 10 columns simultaneously.

- In **BigQuery**: partition by ONE column only (DATE/TIMESTAMP/INT64 range).
- In **SQL Server / PostgreSQL**: partition function based on ONE column's range.
- You can **cluster** (sort) by up to 4 columns in BigQuery — complementary to partitioning.
- For 10,000 rows — partitioning is not beneficial; it's designed for very large tables (millions+ rows). The overhead outweighs the benefit at small scale.

```
-- BigQuery: partition by date, cluster by up to 4 columns
CREATE TABLE dataset.employees
PARTITION BY DATE(hire_date)      -- ONE column for partitioning
CLUSTER BY department, region    -- up to 4 columns for clustering
AS SELECT * FROM dataset.emp_staging;
```

Q: Full outer join vs cross join — difference.

Feature	FULL OUTER JOIN	CROSS JOIN
Output rows	All rows from both tables; NULLs where no match	Every combination of rows (Cartesian)
Row count	Up to A rows + B rows	A rows × B rows
ON clause	Required (matching condition)	Not used

NULLs	Yes — where no matching row on either side	No NULLs (all combinations)
Use case	Find all records including unmatched	Generate all combinations (calendar, test data)
Example	A=3 rows, B=3 rows → max 6 rows	A=3 rows, B=3 rows → 9 rows

```
-- Full Outer Join
SELECT a.id, a.name, b.salary
FROM table_a a
FULL OUTER JOIN table_b b ON a.id = b.id;
-- Includes matched rows + unmatched from A (NULL b cols) + unmatched from B (NULL a cols)

-- Cross Join
SELECT a.product, b.size
FROM products a
CROSS JOIN sizes b;
-- Every product paired with every size – Cartesian product
```

Q: What is data linkage?

Data Linkage (also called record linkage or entity resolution) is the process of **identifying and connecting records from different data sources that refer to the same real-world entity** — even when there is no common unique identifier.

- **Deterministic linkage:** Exact match on known unique keys (customer_id, email, national ID).
- **Probabilistic linkage:** Fuzzy matching on name + DOB + postcode when no exact key exists — uses similarity scores.
- **Use cases:** Merge customer records from CRM and billing system; link patient records across hospitals; deduplicate supplier lists.
- **GCP tools:** Dataflow + custom matching logic; Vertex AI for ML-based matching.

Q: Write SQL query: employees with same salary AND same name (find duplicates).

```
-- Table: emp (emp_id, e_name, salary)

-- Find employees sharing the exact same name AND salary
SELECT
    e_name,
    salary,
    COUNT(*) AS duplicate_count
FROM emp
GROUP BY e_name, salary
HAVING COUNT(*) > 1;

-- Show ALL rows for those duplicate name+salary combinations
SELECT e.*
FROM emp e
JOIN (
    SELECT e_name, salary
    FROM emp
    GROUP BY e_name, salary
    HAVING COUNT(*) > 1
) dups
ON e.e_name = dups.e_name
AND e.salary = dups.salary
ORDER BY e.e_name, e.salary;
```

SECTION E: GCP / Cloud — Missing Topics

Q: What is Cloud Composer — explain briefly.

Cloud Composer is Google Cloud's **fully managed Apache Airflow service**. It removes the need to install, configure and maintain Airflow infrastructure — GCP handles the scheduler, webserver, metadata DB and

worker nodes.

Component	GCP implementation
Airflow Scheduler	Runs inside a GKE cluster managed by GCP
Airflow Webserver	Hosted and accessible via Cloud IAP for secure access
Metadata Database	Cloud SQL (PostgreSQL) — stores DAG state, task history
Workers / Executor	Celery Executor — tasks run on GKE pods
DAG storage	DAGs deployed to a GCS bucket; Composer syncs automatically
Logs	Written to GCS bucket; viewable in Airflow UI

- Supports Cloud IAM for access control — no password management needed.
- Auto-scales workers based on task queue depth.
- Integrates natively with BigQuery, GCS, Dataflow, Dataproc operators.
- Environment versions: Composer 1 (Airflow 1.x), Composer 2 (Airflow 2.x — recommended).

Q: Dataflow — machines/workers behind it and optimisation techniques.

Machines behind Dataflow: Dataflow runs on **Compute Engine VMs** (worker nodes) managed automatically. Workers are launched in your GCP project inside a VPC network.

- **Worker types:** Standard VMs (n1-standard-4 default) or custom machine types.
- **Auto-scaling:** Dataflow monitors throughput and adds/removes workers dynamically based on backlog.
- **Shuffle service:** Managed Dataflow Shuffle offloads group-by and join operations to a persistent backend — workers process data, not shuffle it.
- **Streaming Engine:** Offloads windowing/state management from workers to Google infrastructure — fewer workers needed for streaming pipelines.

Dataflow optimisation techniques:

- **Enable Dataflow Shuffle** (batch): `--experiments=shuffle_mode=service` — reduces worker memory and speeds up group-by operations.
- **Enable Streaming Engine**: `--enable_streaming_engine` — reduces VM footprint for streaming pipelines.
- **Use Dataflow templates**: Pre-built, tested pipelines — no recompilation needed.
- **Combine transforms**: Use `Combine` instead of `GroupByKey` where possible — `Combine` does partial aggregation per worker before shuffling.
- **Avoid heavy Python UDFs**: Use Java SDK or built-in transforms where possible — Python has serialisation overhead.
- **Right-size workers**: Choose machine type based on pipeline profile (CPU-heavy → more vCPU; state-heavy → more memory).
- **Use ParDo with DoFn lifecycle**: `setup()` for one-time initialisation (e.g., load model) rather than per-element.

Q: How to create a partition table in BigQuery (DDL)?

```
-- █ By DATE column █  
CREATE OR REPLACE TABLE `project.dataset.sales`  
(  
    sale_id          INT64,  
    sale_date        DATE,  
    customer_id      STRING,  
    product_id       STRING,  
    region           STRING,  
    amount           FLOAT64  
)  
PARTITION BY sale_date  
CLUSTER BY region, product_id  
OPTIONS (  
    partition_expiration_days = 365 -- auto-delete partitions older than 1 year
```



```

        content_type="application/json",
    )
    print(f"Log written to {blob.name}")

```

Q: Do you have experience with other clouds besides GCP (AWS / Azure)?

Sample answer for GCP-focused engineers:

- **Primary experience:** GCP — BigQuery, GCS, Pub/Sub, Dataflow, Composer, Dataproc, IAM.
- **AWS equivalents awareness:** S3 ≈ GCS, Redshift ≈ BigQuery, Glue ≈ Dataflow, MWAA ≈ Composer, Kinesis ≈ Pub/Sub, EMR ≈ Dataproc.
- **Azure equivalents awareness:** ADLS ≈ GCS, Synapse ≈ BigQuery, ADF ≈ Dataflow, Event Hubs ≈ Pub/Sub.
- Most data engineering concepts (ETL, partitioning, streaming) are transferable across clouds — the services differ in naming and specific APIs.

Q: What is SSID and Control Flow? (HSBC interview context)

SSID (Session ID / SQL Server Integration Services ID): In SSIS (SQL Server Integration Services), each package execution gets a session ID. More commonly in SQL Server context, SSID refers to the unique identifier of an Integration Services package instance.

Control Flow in SSIS / ETL tools: The **Control Flow** tab defines the **overall workflow** of a package — which tasks run, in what order, and under what conditions. It orchestrates tasks like SQL tasks, data flow tasks, script tasks, loops, and conditional logic.

Concept	In SSIS	GCP equivalent
Control Flow	Orchestrates task execution order	Airflow DAG
Data Flow	Moves and transforms data	Dataflow pipeline
Package	Unit of deployment	DAG file (.py)
Connection Manager	Stores connection strings	Airflow Connection
Variables	Runtime parameters	Airflow Variables / XCom

Q: What are actuators in Python? (HCL interview context)

Note: 'Actuators' is not a standard Python term. The interviewer likely meant one of the following:

- **Context managers** — objects using `__enter__` / `__exit__` (the 'with' statement).
- **Decorators** — functions that modify/wrap other functions.
- **Generators / Iterators** — objects with `__iter__` and `__next__`.
- **Spring Boot Actuators context:** If from a Java/Spring background, Actuators are monitoring endpoints — not a Python concept.
- **Best interview answer:** Clarify the question — 'Could you clarify what you mean by actuators in Python? Are you referring to decorators, context managers, or generators?'

```

# Context Manager (most likely what was meant)
class DatabaseConnection:
    def __enter__(self):
        self.conn = connect_to_db()
        return self.conn

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.conn.close() # always runs on exit
        return False      # do not suppress exceptions

# Usage
with DatabaseConnection() as conn:
    conn.execute("SELECT * FROM employees")

```

Q: write_truncate vs write_append vs write_empty in BigQuery.

Mode	Behaviour	Use case
WRITE_TRUNCATE	Deletes ALL existing rows then writes new data	Full refresh / reload
WRITE_APPEND	Adds new rows to existing data (no delete)	Incremental / daily append
WRITE_EMPTY	Writes ONLY if table is empty; fails if data exists	First-time load safety check

```
from google.cloud import bigquery

client = bigquery.Client()
table_id = "project.dataset.orders"

job_config = bigquery.LoadJobConfig(
    source_format=bigquery.SourceFormat.CSV,
    skip_leading_rows=1,
    autodetect=True,

    # Choose ONE of:
    write_disposition=bigquery.WriteDisposition.WRITE_TRUNCATE, # full refresh
    # write_disposition=bigquery.WriteDisposition.WRITE_APPEND, # incremental
    # write_disposition=bigquery.WriteDisposition.WRITE_EMPTY, # first load only
)

load_job = client.load_table_from_uri(
    "gs://bucket/orders/*.csv",
    table_id,
    job_config=job_config,
)

load_job.result()
```

Q: Can't we transform at source and load directly into BigQuery? (no staging)

- **Source system overload:** Running heavy transformations on Oracle/RDBMS consumes production database resources — risks impacting live users.
- **Lost raw data:** If you transform at source, the original raw data is gone. You cannot reprocess or audit what was received.
- **No reprocessing:** If the BQ load fails mid-way, you must re-query the source system (which may have updated since). With GCS staging you just re-load the file.
- **Schema mismatches:** GCS staging lets you validate and handle schema changes before they corrupt the target table.
- **Compliance:** Many regulatory frameworks (SOX, GDPR) require raw data retention in an immutable landing zone.
- **Performance:** GCS staging enables parallel loading of multiple files to BigQuery, which is faster than streaming row by row from source.

SECTION F: Open-ended / Behavioural Questions

Q: What is the volume of data you handle in your project?

Sample structured answer framework:

- **Daily ingestion:** 'We ingest approximately X GB / TB of data daily from [N] source systems.'
- **Total data lake size:** 'Our GCS data lake holds around X TB of raw and processed data.'
- **BigQuery tables:** 'Our largest fact tables contain approximately X billion rows, partitioned by date.'
- **Pipeline frequency:** 'We run [hourly/daily/real-time] pipelines processing [N] files per run, each approximately [size] MB.'
- **Streaming:** 'Real-time Pub/Sub pipelines handle approximately X events/second.'

Q: What are the performance challenges you have faced?

- **Data skew:** Certain partition keys had disproportionately large data (e.g., top 5 customers = 60% of rows). Fixed with salting technique in Spark.
- **Slow BigQuery queries:** Full table scans due to missing partition filters. Fixed by ensuring all queries filter on the partition column.
- **Airflow DAG backlog:** catchup=True created 30 days of backlogged runs all executing simultaneously. Fixed by setting catchup=False and doing a controlled manual backfill.
- **OOM in Spark:** Collecting large DataFrames to driver node. Fixed by writing to GCS instead of using collect().
- **Schema drift:** Source system added columns without notice, breaking the BQ load job. Fixed by adding autodetect + schema evolution handling.
- **API rate limits:** Source API throttled after N requests/minute. Fixed by adding exponential backoff retry logic.

Q: Do you know about third-party tools outside GCP?

- **Workflow:** Apache Airflow (open source), Prefect, Dagster.
- **Transformation:** dbt Core / dbt Cloud — SQL-first transformation.
- **Data quality:** Great Expectations, Soda Core.
- **Version control:** GitHub, GitLab, Bitbucket.
- **CI/CD:** GitHub Actions, Jenkins, CircleCI.
- **Monitoring:** Grafana, Datadog, PagerDuty.
- **Data catalogue:** Collibra, Alation (complement to GCP Data Catalog).
- **ETL tools:** Informatica, Talend, Fivetran, Airbyte.

Q: How do you schedule DAG execution and handle the schedule?

Covered in Section B. Additional tips:

- Use **UTC timezone** for all `schedule_interval` values to avoid DST issues.
- Set **start_date** to a fixed past date — never `datetime.now()` (causes non-deterministic behaviour).
- Use **data_interval_start / data_interval_end** template variables (Airflow 2.x) instead of deprecated `execution_date`.
- For event-driven triggering use the **Airflow REST API** or **Cloud Functions** → **trigger DAG via API**.